
Introduction to Parallel Programming

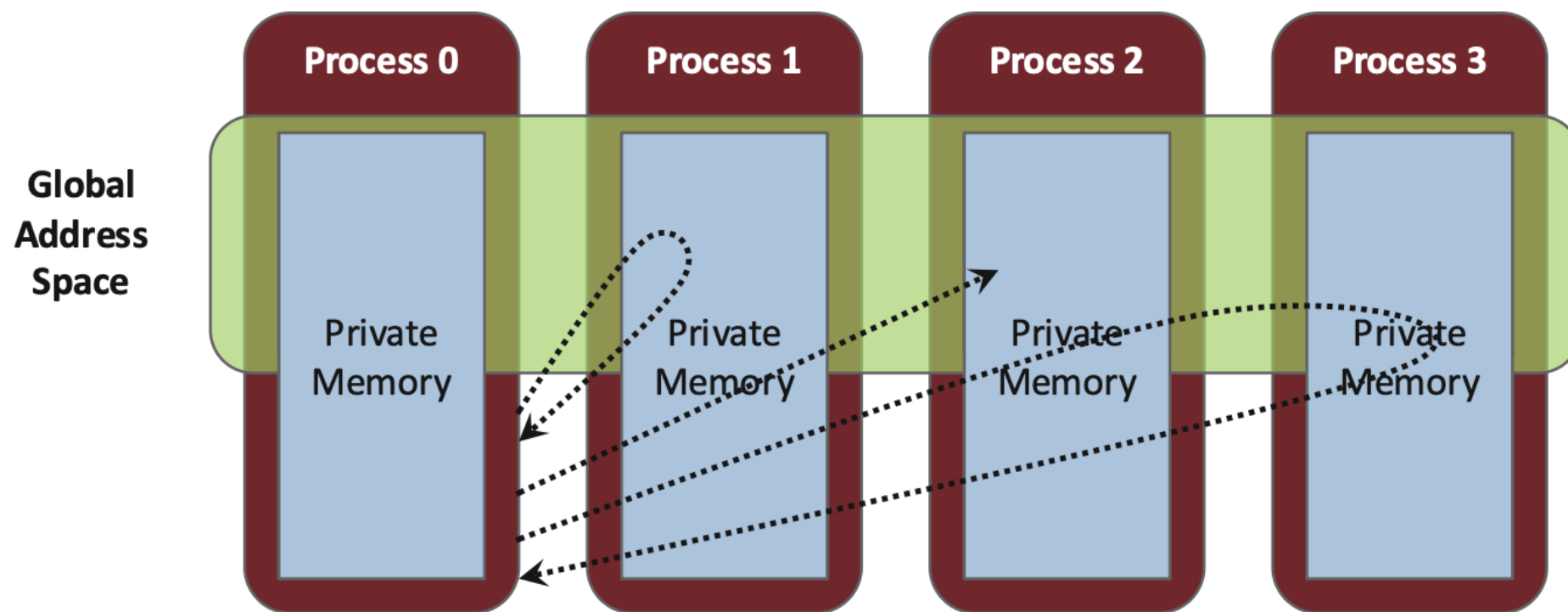
Lecture xx: MPI One-sided Communication

谭光明

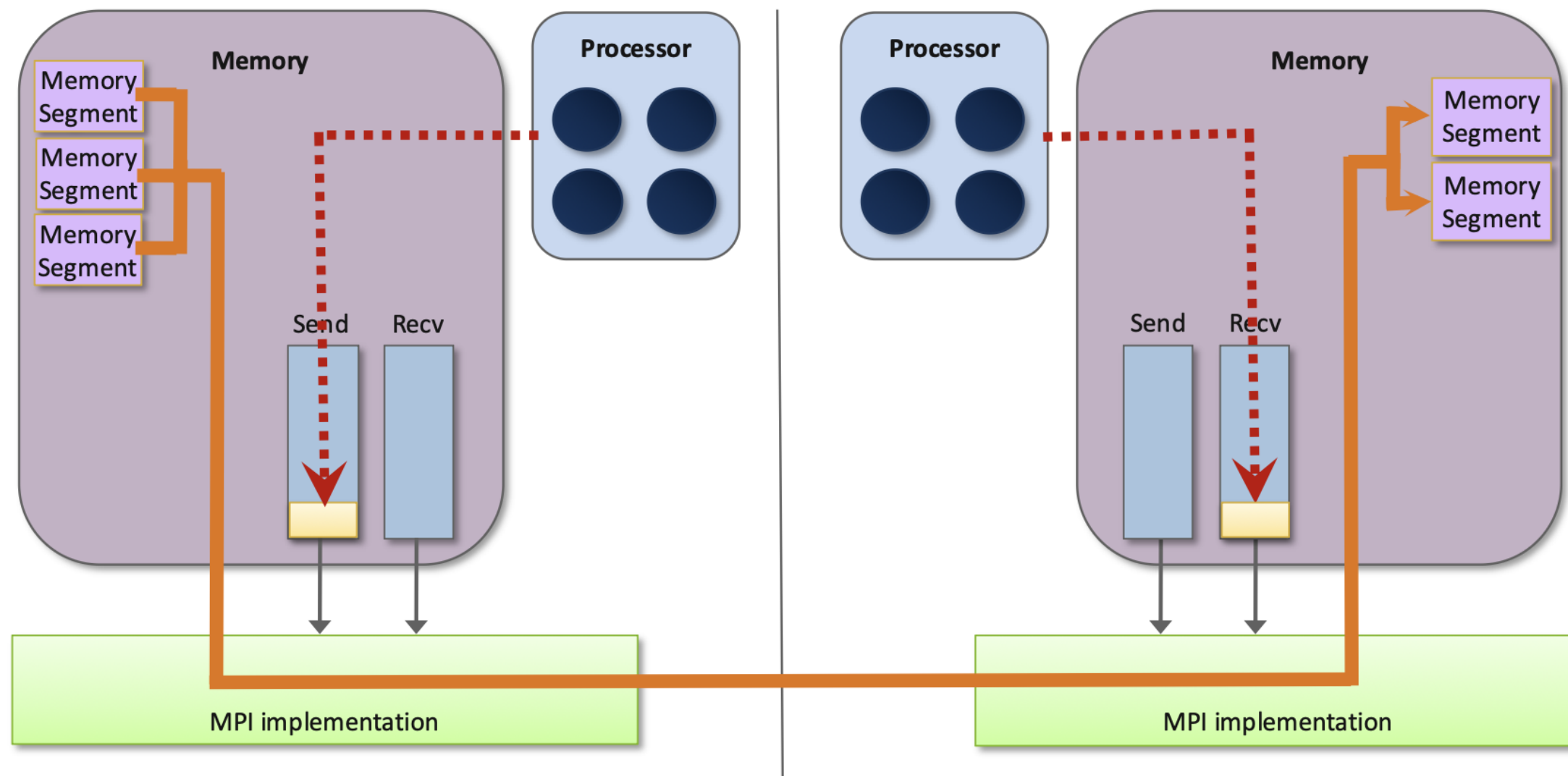
高性能计算机研究中心

One-sided Communication

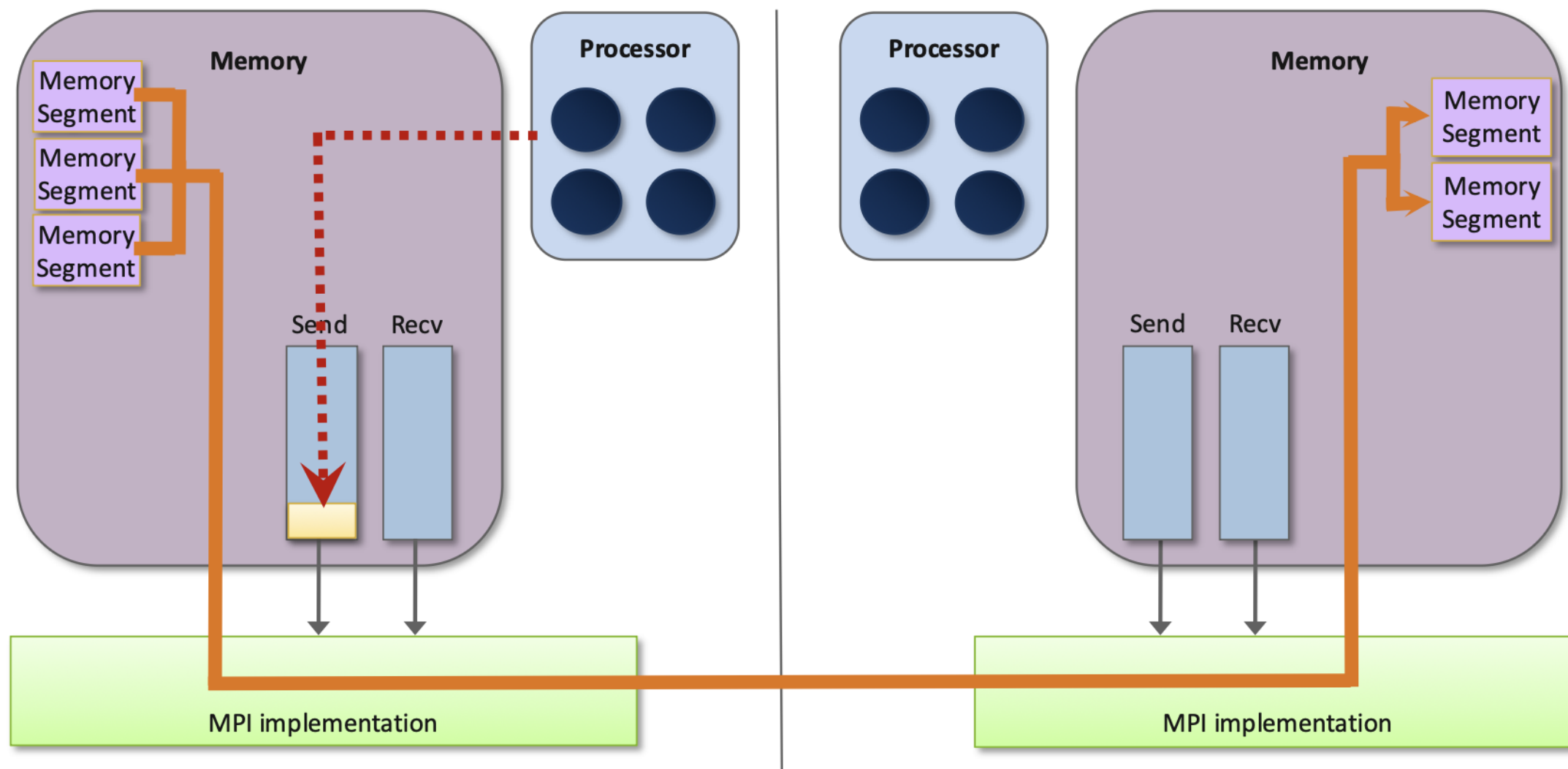
- The basic idea of one-sided communication models is to decouple data movement with process synchronization
 - Should be able to move data without requiring that the remote process synchronize
 - Each process exposes a part of its memory to other processes
 - Other processes can directly read from or write to this memory



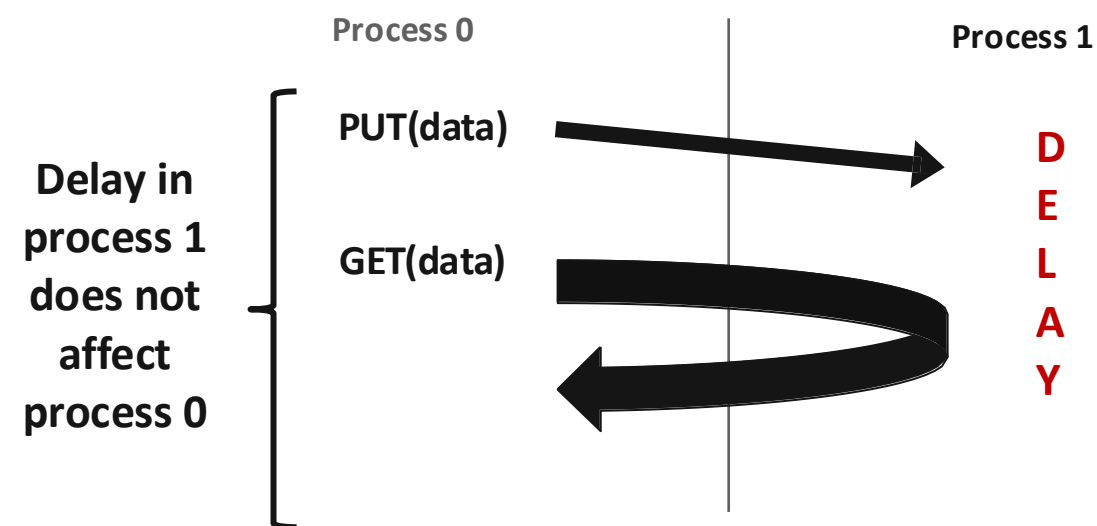
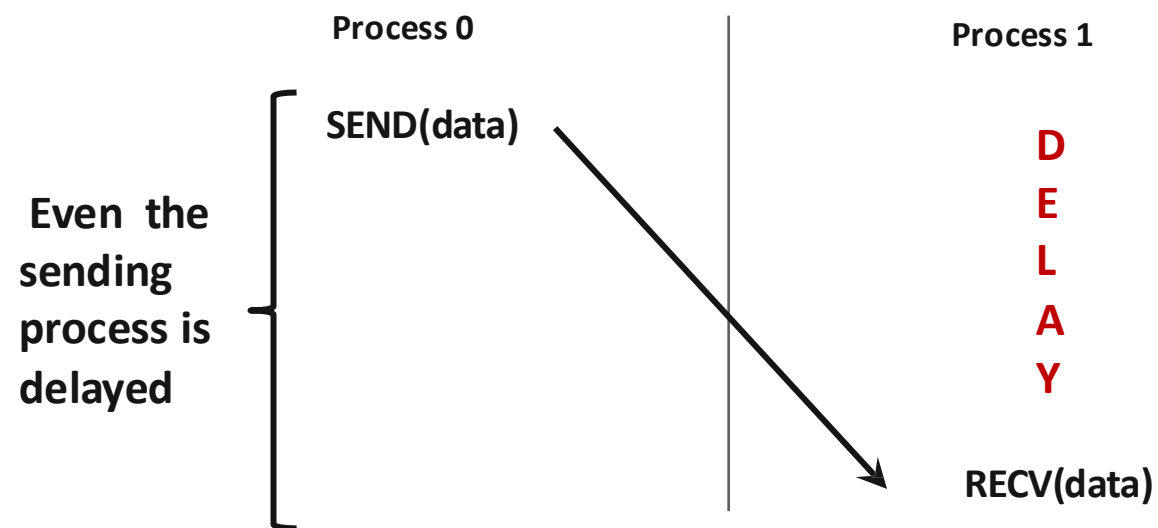
Two-sided Communication Example



One-sided Communication Example



Comparing One-sided and Two-sided Programming



MPI RMA is Carefully and Precisely Specified

- To work on both cache-coherent and non-cache-coherent systems
 - Even though there aren't many non-cache-coherent systems, it is designed with the future in mind
- There even exists a formal model for MPI-3 RMA that can be used by tools and compilers for optimization, verification, etc.
 - See "Remote Memory Access Programming in MPI-3" by Hoefler, Dinan, Thakur, Barrett, Balaji, Gropp, Underwood. ACM TOPC, July 2015.
<https://dl.acm.org/doi/10.1145/2780584>
 - <http://htr.inf.ethz.ch/publications/index.php?pub=201>

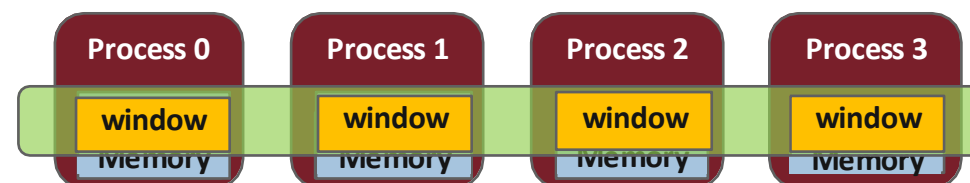
What we need to know in MPI RMA

- How to create remote accessible memory?
- How to read, writing and update remote memory
- How to ensure that data is available - Data Synchronization
- How RMA operations interact with processes and other operations - Memory Model

Creating Public Memory

- Any memory used by a process is, by default, only locally accessible

- `X = malloc(100);`



- Once the memory is allocated, the user has to make an explicit MPI call to declare a memory region as remotely accessible
- MPI terminology for remotely accessible memory is a "window"
- A group of processes collectively create a "window"
- Once a memory region is declared as remotely accessible, all processes in the window can read/write/update data to this memory without explicitly synchronizing with the target process
- MPI routines usually required for read/write/update of remote data

Window creation models

- Four models exist
 - `MPI_WIN_ALLOCATE`
 - You want to create a buffer and directly make it remotely accessible
 - `MPI_WIN_CREATE`
 - You already have an allocated buffer that you would like to make remotely accessible
 - `MPI_WIN_CREATE_DYNAMIC`
 - You don't have a buffer yet, but will have one in the future
 - You may want to dynamically add/remove buffers to/from the window
 - `MPI_WIN_ALLOCATE_SHARED`
 - You want multiple processes on the same node* to share a buffer

* More precisely, processes that can share memory, usually through hardware and operating system support. Often true if the processes are on the same node.

MPI_WIN_ALLOCATE

```
MPI_Win_allocate(MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm,  
                 void *baseptr, MPI_Win *win)
```

- **Create a remotely accessible memory region in an RMA window**
 - Only data exposed in a window can be accessed with RMA ops.
- **Arguments:**
 - **size** - size of local data in bytes (nonnegative integer)
 - **disp_unit** - local unit size for displacements, in bytes (positive integer)
 - **info** - info argument (handle)
 - **comm** - communicator (handle)
 - **baseptr** - pointer to exposed local data
 - **win** - window (handle)

Example with MPI_WIN_ALLOCATE

```
int main(int argc, char ** argv)
{
    int *a;
    MPI_Win win;

    MPI_Init(&argc, &argv);

    /* collectively create remote accessible memory in a window */
    MPI_Win_allocate(1000*sizeof(int), sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD, &a, &win);

    /* Array 'a' is now accessible from all processes in
     * MPI_COMM_WORLD */

    MPI_Win_free(&win);

    MPI_Finalize(); return 0;
}
```

MPI_WIN_CREATE

```
MPI_Win_create(void *base, MPI_Aint size, int disp_unit, MPI_Info info, MPI_Comm comm,  
               MPI_Win *win)
```

- **Expose a region of memory in an RMA window**
 - Only data exposed in a window can be accessed with RMA ops.
- **Arguments:**
 - **base** - pointer to local data to expose
 - **size** - size of local data in bytes (nonnegative integer)
 - **disp_unit** - local unit size for displacements, in bytes (positive integer)
 - **info** - info argument (handle)
 - **comm** - communicator (handle)
 - **win** - window (handle)

Example with MPI_WIN_CREATE

```
int main(int argc, char ** argv)
{
    int *a;      MPI_Win win;

    MPI_Init(&argc, &argv);

    /* create private memory */
    a = (int *) malloc(1000 * sizeof(int));
    /* use private memory like you normally would */
    for (int i = 0; i < 1000; i++) a[i] = i + 1;

    /* collectively declare memory as remotely accessible */
    MPI_Win_create(a, 1000*sizeof(int), sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD, &win);

    /* Array 'a' is now accessible by all processes in
     * MPI_COMM_WORLD */

    MPI_Win_free(&win);
    free(a);
    MPI_Finalize(); return 0;
}
```

MPI_WIN_CREATE_DYNAMIC

```
MPI_Win_create_dynamic(MPI_Info info, MPI_Comm comm, MPI_Win *win)
```

- Create an RMA window, to which data can later be attached
 - Only data exposed in a window can be accessed with RMA ops
- Initially “empty”
 - Application can dynamically attach/detach memory to this window by calling `MPI_Win_attach/detach`
 - Application can access data on this window only after a memory region has been attached
- Window origin is `MPI_BOTTOM`
 - Displacements are segment addresses relative to `MPI_BOTTOM`
 - Must tell others the displacement after calling `attach`

Example with MPI_WIN_CREATE_DYNAMIC

```
int main(int argc, char ** argv)
{
    int *a;      MPI_Win win;

    MPI_Init(&argc, &argv);  MPI_Win_create_dynamic(MPI_INFO_NULL, MPI_COMM_WORLD, &win);

    /* create private memory */
    a = (int *) malloc(1000 * sizeof(int));
    /* use private memory like you normally would */
    for (int i = 0; i < 1000; i++) a[i] = i + 1;

    /* locally declare memory as remotely accessible */ MPI_Win_attach(win, a,
1000*sizeof(int));

    /* Array 'a' is now accessible from all processes */

    /* undeclare remotely accessible memory */ MPI_Win_detach(win, a);
        free(a); MPI_Win_free(&win);

    MPI_Finalize(); return 0;
}
```

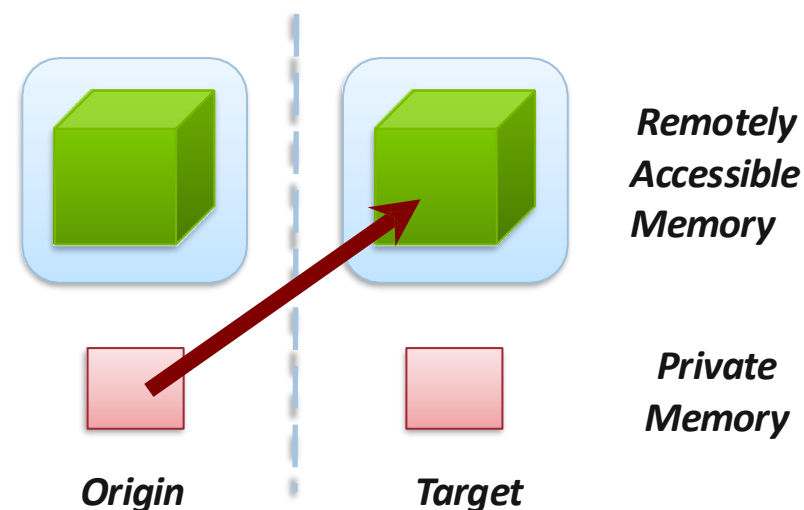
Data movement

- MPI provides ability to read, write and atomically update (modify) data in remotely accessible memory regions
 - `MPI_PUT`
 - `MPI_GET`
 - `MPI_ACCUMULATE` `(atomic)`
 - `MPI_GET_ACCUMULATE` `(atomic)`
 - `MPI_COMPARE_AND_SWAP` `(atomic)`
 - `MPI_FETCH_AND_OP` `(atomic)`
- There are additional routines which provide another data synchronization option (See `MPI_RPUT` etc.)

Data movement: *Put*

```
MPI_Put(const void *origin_addr, int origin_count, MPI_Datatype origin_dtype,  
        int target_rank, MPI_Aint target_disp, int target_count,  
        MPI_Datatype target_dtype, MPI_Win win)
```

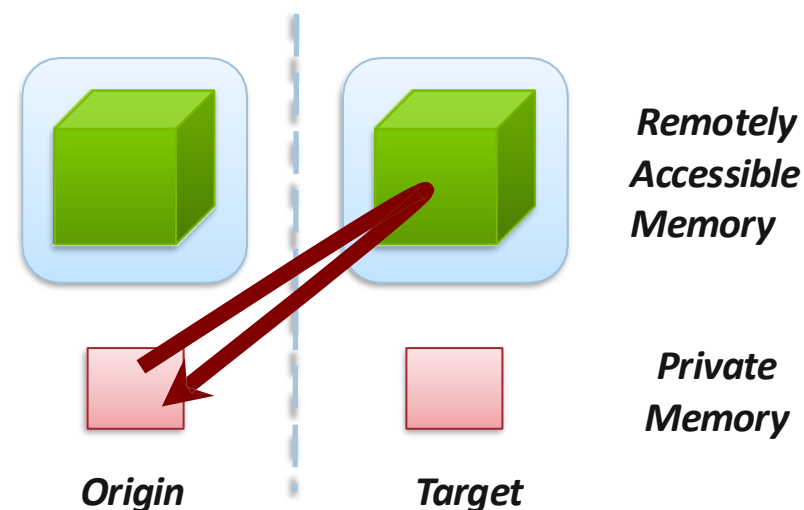
- Move data from origin, to target
- Separate data description triples for **origin** and **target**



Data movement: *Get*

```
MPI_Get(void *origin_addr, int origin_count, MPI_Datatype origin_dtype,  
        int target_rank, MPI_Aint target_disp, int target_count,  
        MPI_Datatype target_dtype, MPI_Win win)
```

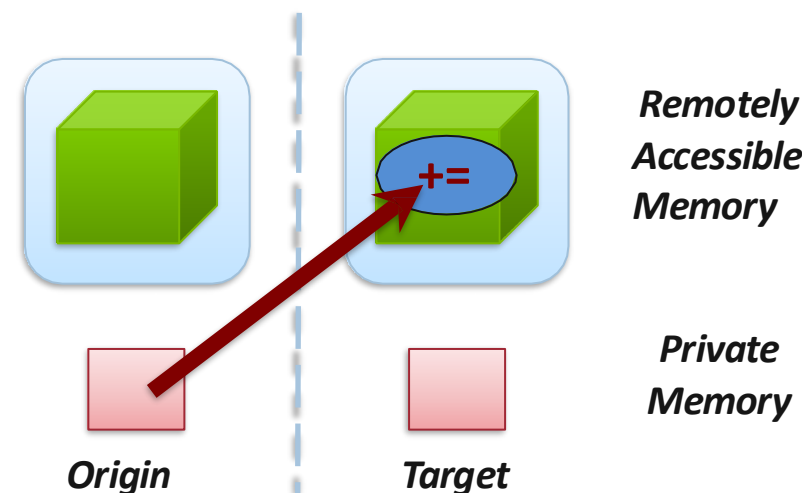
- Move data to origin, from target
- Separate data description triples for **origin** and **target**



Atomic Data Aggregation: *Accumulate*

```
MPI_Accumulate(const void *origin_addr, int origin_count, MPI_Datatype origin_dtype,  
              int target_rank, MPI_Aint target_disp, int target_count,  
              MPI_Datatype target_dtype, MPI_Op op, MPI_Win win)
```

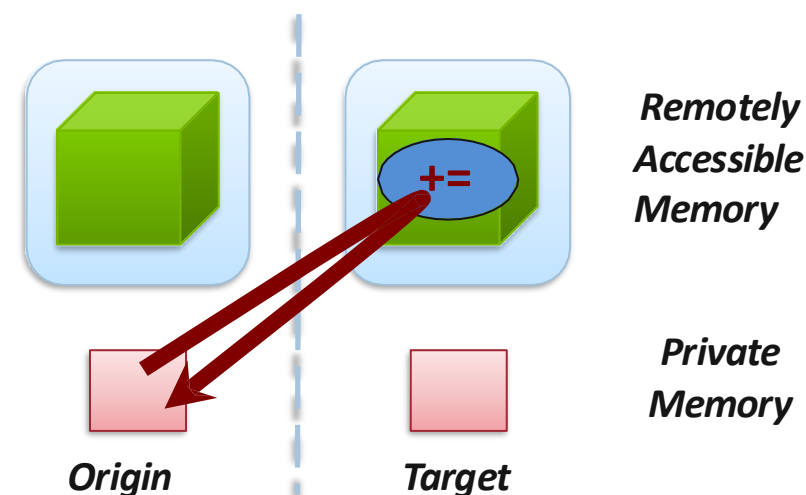
- Atomic update operation, similar to a put
 - Reduces origin and target data into target buffer using op argument as combiner
 - Op = MPI_SUM, MPI_PROD, MPI_OR, MPI_REPLACE, MPI_NO_OP, ...
 - Predefined ops only, no user-defined operations
- Different data layouts between target/origin OK
 - Basic type elements must match
- Op = MPI_REPLACE
 - Implements $f(a,b)=b$
 - Atomic PUT



Atomic Data Aggregation: *Get Accumulate*

```
MPI_Get_accumulate(const void *origin_addr, int origin_count, MPI_Datatype origin_dtype,  
                  void *result_addr, int result_count, MPI_Datatype result_dtype,  
                  int target_rank, MPI_Aint target_disp, int target_count,  
                  MPI_Datatype target_dtype, MPI_Op op, MPI_Win win)
```

- Atomic read-modify-write
 - Op = MPI_SUM, MPI_PROD, MPI_OR, MPI_REPLACE, MPI_NO_OP, ...
 - Predefined ops only
- Result stored in target buffer
- Original data stored in result buf
- Different data layouts between target/origin OK
 - Basic type elements must match
- Atomic get with MPI_NO_OP
- Atomic swap with MPI_REPLACE



Atomic Data Aggregation: *FOP* and *CAS*

```
MPI_Fetch_and_op(const void *origin_addr, void *result_addr, MPI_Datatype dtype,  
                int target_rank, MPI_Aint target_disp, MPI_Op op, MPI_Win win)
```

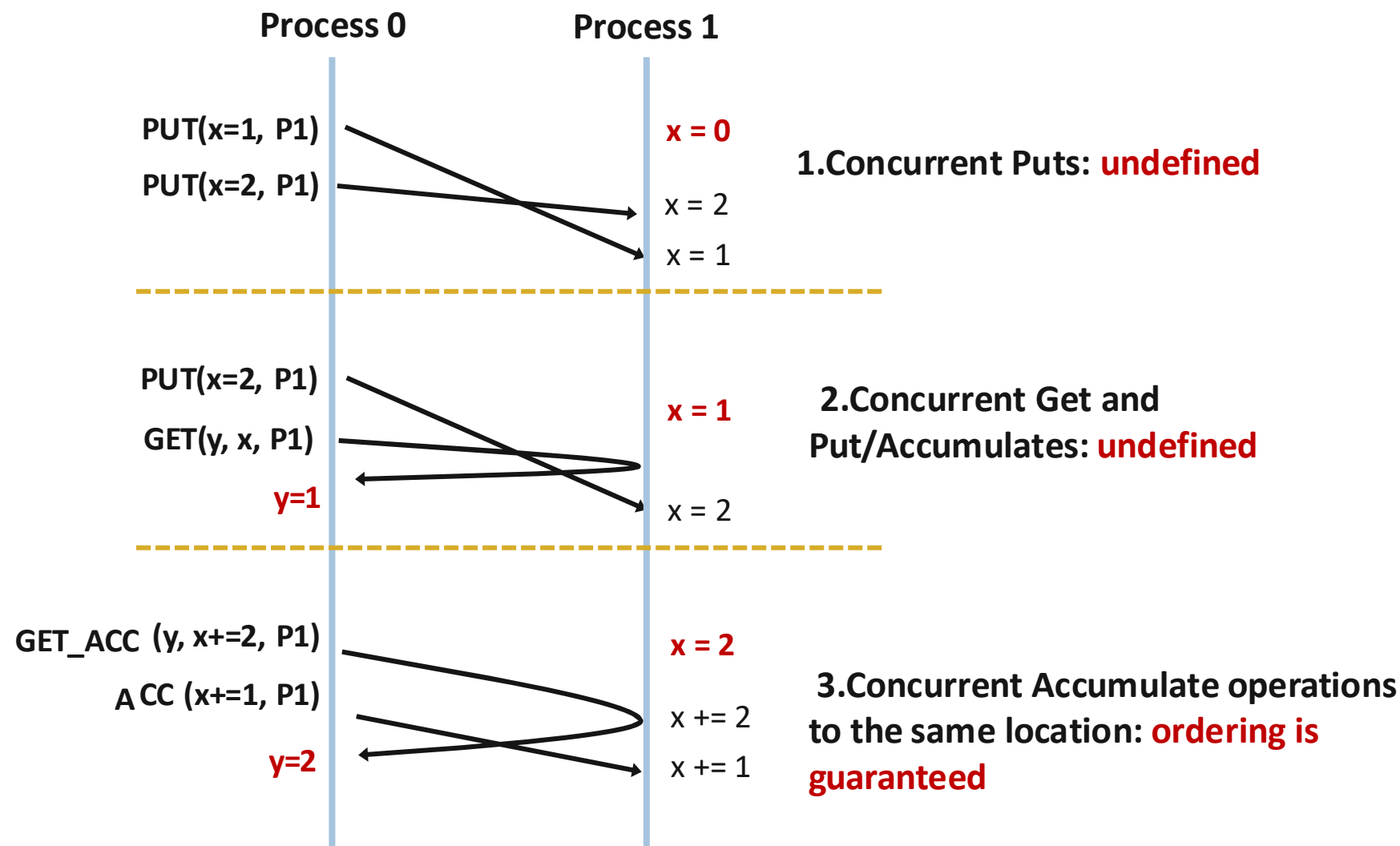
```
MPI_Compare_and_swap(const void *origin_addr, const void *compare_addr,  
                    void *result_addr, MPI_Datatype dtype, int target_rank,  
                    MPI_Aint target_disp, MPI_Win win)
```

- **FOP: Simpler version of MPI_Get_accumulate**
 - All buffers share a single predefined datatype
 - No count argument (it's always 1)
 - Simpler interface allows hardware optimization
- **CAS: Atomic swap if target value is equal to compare value**

Ordering of Operations in MPI RMA

- No guaranteed ordering for Put/Get operations
- Result of concurrent Puts to the same location undefined
- Result of Get concurrent with Put/Accumulate undefined
 - Can be garbage in both cases
- Result of concurrent accumulate operations to the same location are defined according to the order in which they occurred
 - Atomic put: Accumulate with op = MPI_REPLACE
 - Atomic get: Get_accumulate with op = MPI_NO_OP
- Accumulate operations from a given process are ordered by default
 - User can tell the MPI implementation that (s)he does not require ordering as optimization hint with info when window is created
 - You can ask for only the needed orderings: RAW (read-after-write), WAR, RAR, or WAW

Examples with operation ordering



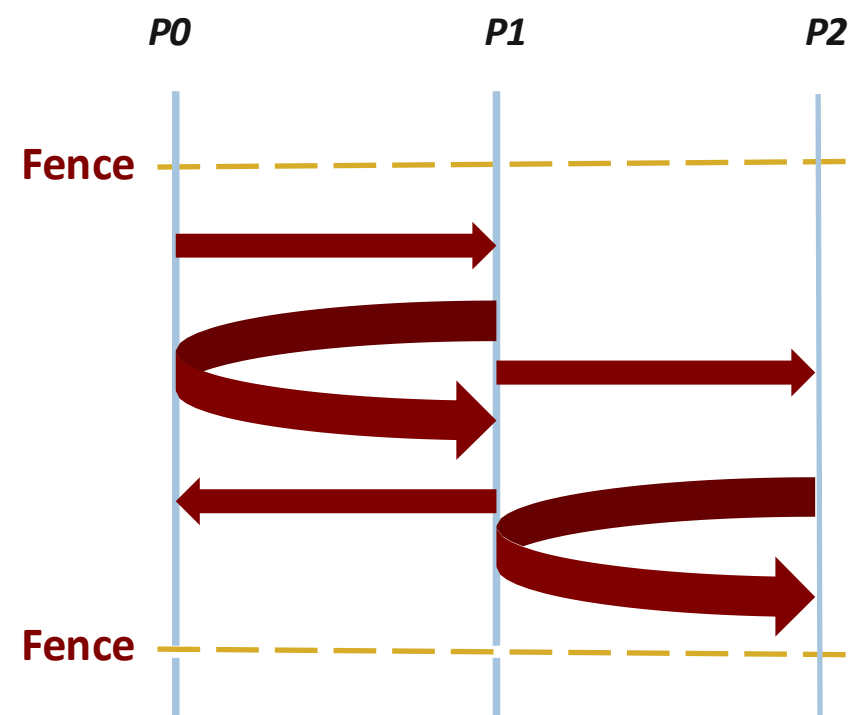
RMA Synchronization Models

- RMA data access model
 - When is a process allowed to read/write remotely accessible memory?
 - When is data written by process X is available for process Y to read?
 - RMA synchronization models define these semantics
- Three synchronization models provided by MPI:
 - Fence (active target)
 - Post-start-complete-wait (generalized active target)
 - Lock/Unlock (passive target)
- Data accesses occur within “epochs”
 - *Access epochs*: contain a set of operations issued by an origin process
 - *Exposure epochs*: enable remote processes to update a target’s window
 - Epochs define ordering and completion semantics
 - Synchronization models provide mechanisms for establishing epochs
 - E.g., starting, ending, and synchronizing epochs

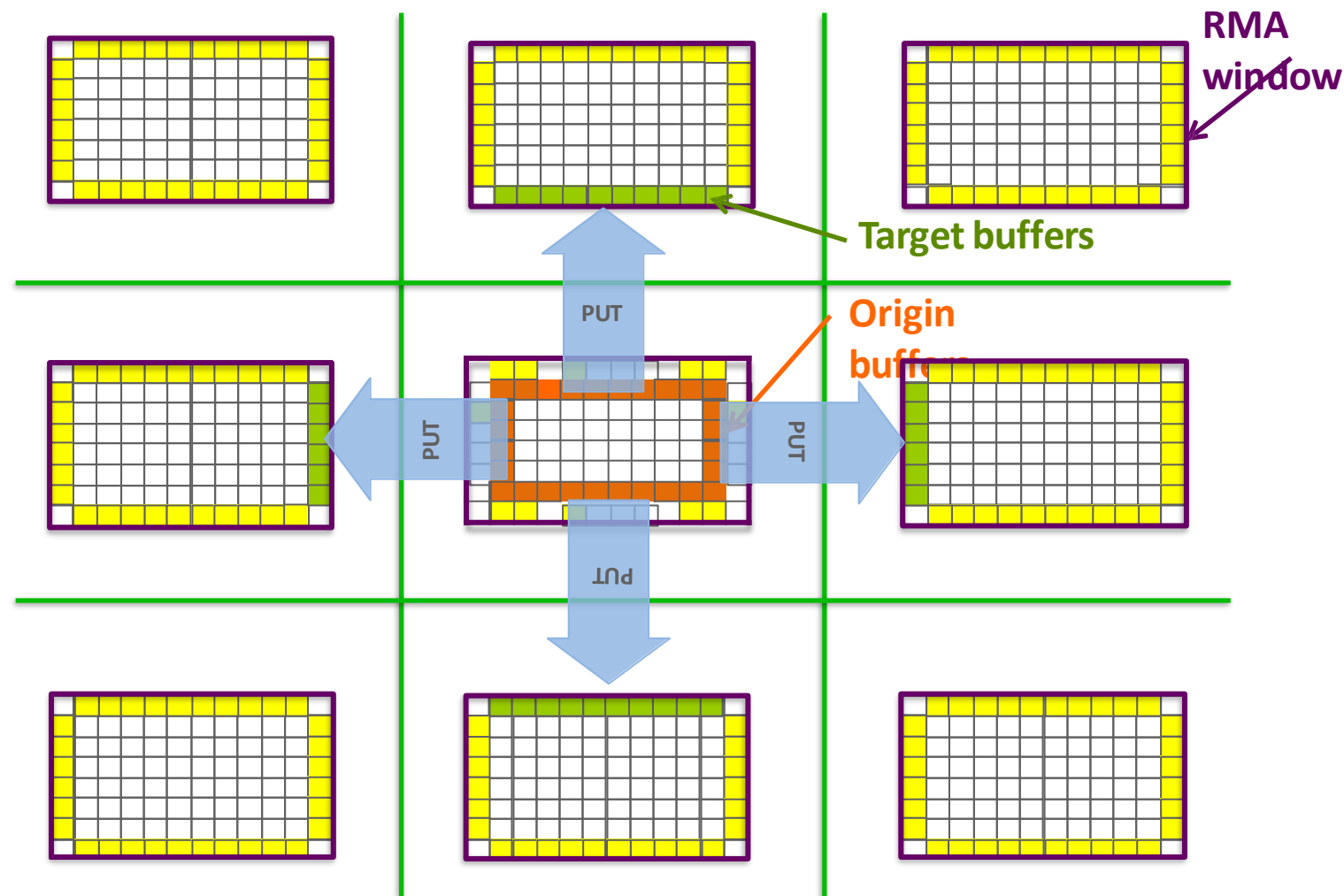
Fence: Active Target Synchronization

```
MPI_Win_fence(int assert, MPI_Win win)
```

- Collective synchronization model
- Starts *and* ends access and exposure epochs on all processes in the window
- All processes in group of “win” do an `MPI_WIN_FENCE` to open an epoch
- Everyone can issue PUT/GET operations to read/write data
- Everyone does an `MPI_WIN_FENCE` to close the epoch
- All operations complete at the second fence synchronization



Exercise 1: Stencil with RMA Fence (1/2)



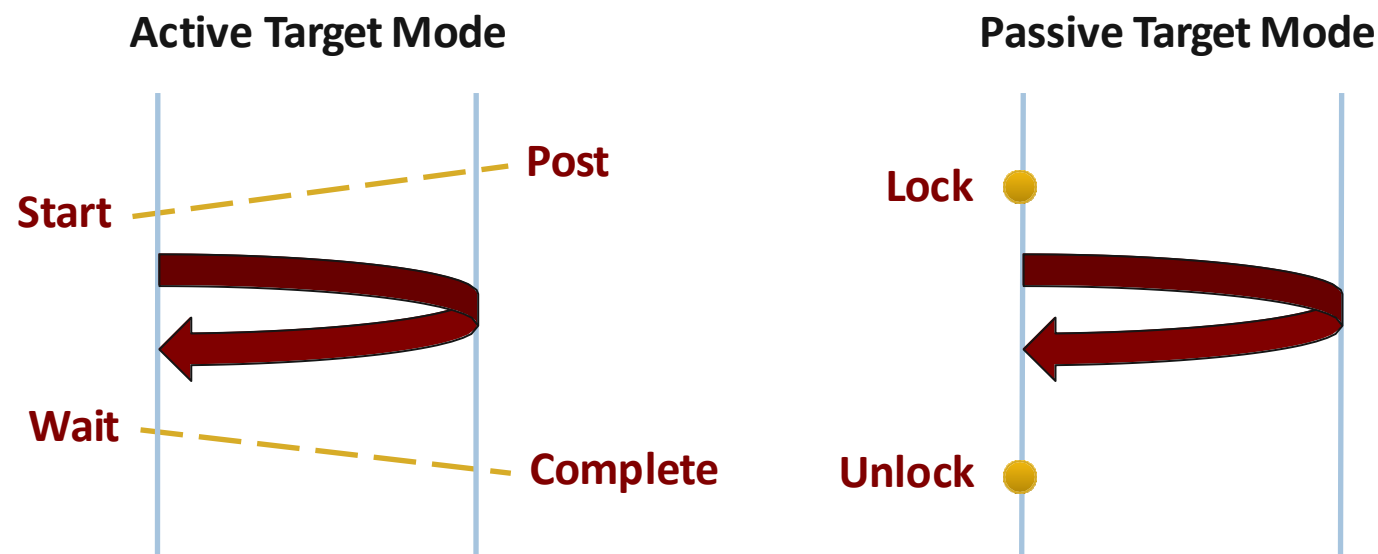
Exercise 1: Stencil with RMA Fence (2/2)

- In the derived datatype version of the stencil code
 - Used nonblocking communication
 - Used derived datatypes
- Let's try to use RMA fence
 - Move data with PUT instead of send/recv
- *Start from derived_datatype/stencil.c*
- *Solution available in rma/stencil_fence_put.c*

Exercise 2: Stencil with RMA Fence (GET model)

- In the derived datatype version of the stencil code
 - Used nonblocking communication
 - Used derived datatypes
- Let's try to use RMA fence
 - Move data with GET instead of send/recv
- *Start from `rma/stencil_fence_put.c`*
- *Solution available in `rma/stencil_fence_get.c`*

Lock/Unlock: Passive Target Synchronization



- **Passive mode: One-sided, *asynchronous* communication**
 - Target does not participate explicitly in communication operation
- **Shared memory-like model (in the sense that the operations are all called on the origin process, with no synchronization routines called on the target process)**
- **Lock is an unfortunate choice of name – not a mutual exclusion; more a begin/end of RMA operations**

Passive Target Synchronization

```
MPI_Win_lock(int locktype, int rank, int assert, MPI_Win win)
```

```
MPI_Win_unlock(int rank, MPI_Win win)
```

```
MPI_Win_flush/flush_local(int rank, MPI_Win win)
```

- **Lock/Unlock: Begin/end passive mode epoch**
 - Target process does not make a corresponding MPI call
 - Can initiate multiple passive target epochs to different processes
 - Concurrent epochs to same process not allowed (affects threads)
- **Lock type**
 - SHARED: Other processes using shared can access concurrently
 - EXCLUSIVE: No other processes can access concurrently
- **Flush: Remotely complete RMA operations to the target process**
 - After completion, data can be read by target process or a different process
- **Flush_local: Locally complete RMA operations to the target process**

Advanced Passive Target Synchronization

```
MPI_Win_lock_all(int assert, MPI_Win win)
```

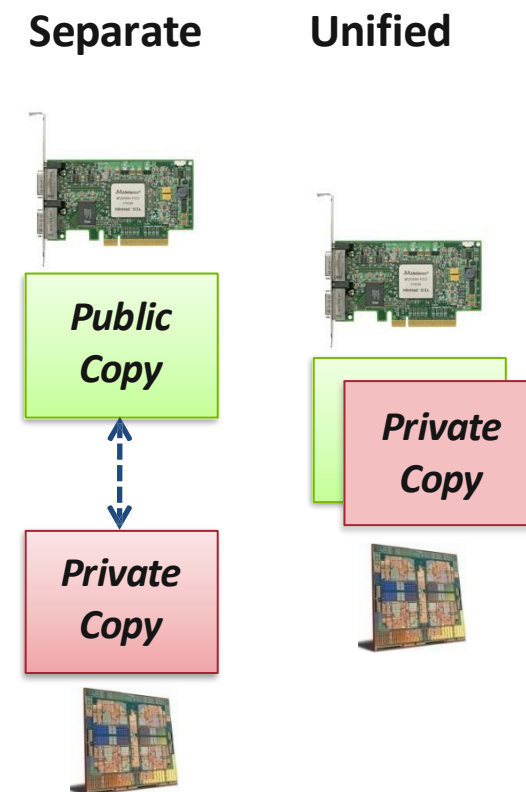
```
MPI_Win_unlock_all(MPI_Win win)
```

```
MPI_Win_flush_all/flush_local_all(MPI_Win win)
```

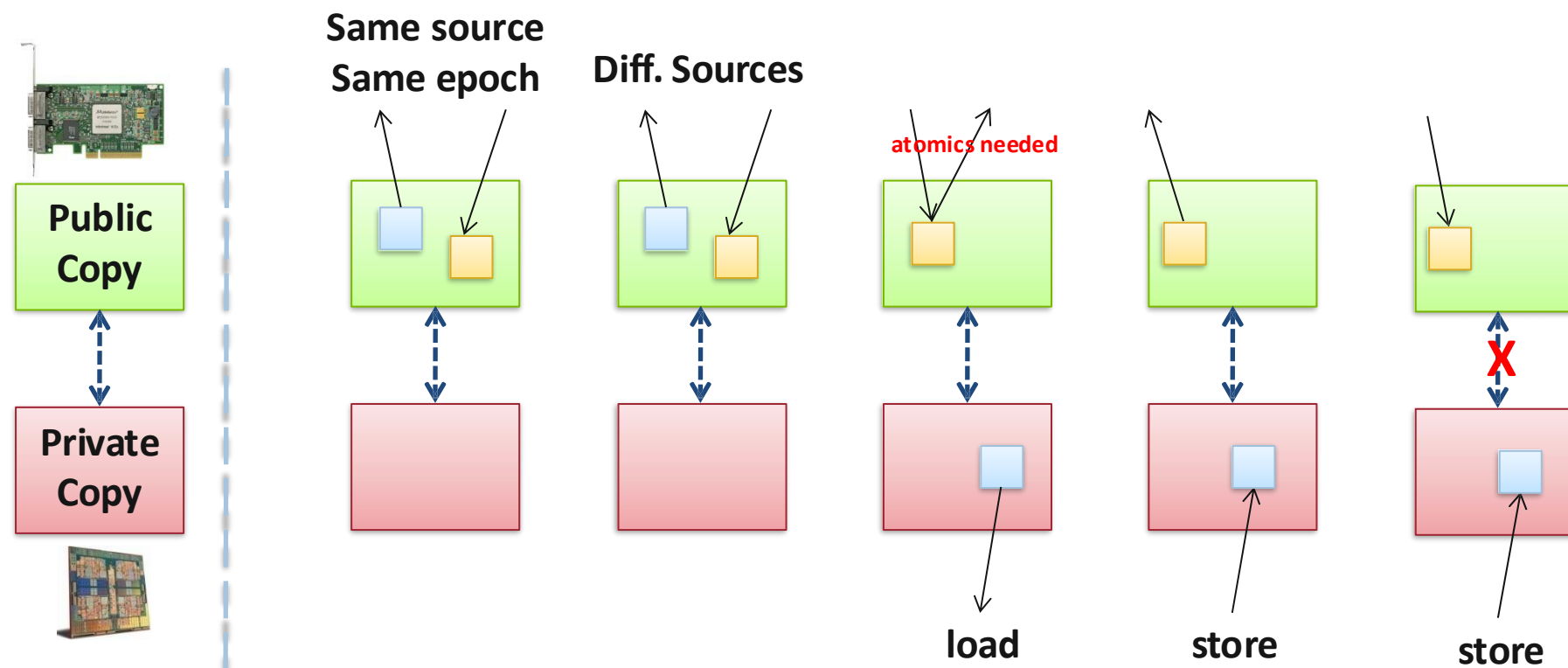
- **Lock_all**: Shared lock, passive target epoch to all other processes in window
 - Expected usage is long-lived: lock_all, put/get, flush, ..., unlock_all
- **Flush_all**: remotely complete RMA operations to all processes in window
- **Flush_local_all**: locally complete RMA operations to all processes in window

MPI RMA Memory Model

- MPI-3 provides two memory models: separate and unified
- Separate Model
 - Logical public and private copies
 - MPI provides software coherence between window copies
 - Extremely portable; works on systems that don't provide hardware coherence
- New (in MPI-3) Unified Model
 - Single copy of the window
 - System must provide coherence
 - Superset of separate semantics
 - E.g., allows concurrent local/remote access
 - Provides access to full performance potential of hardware

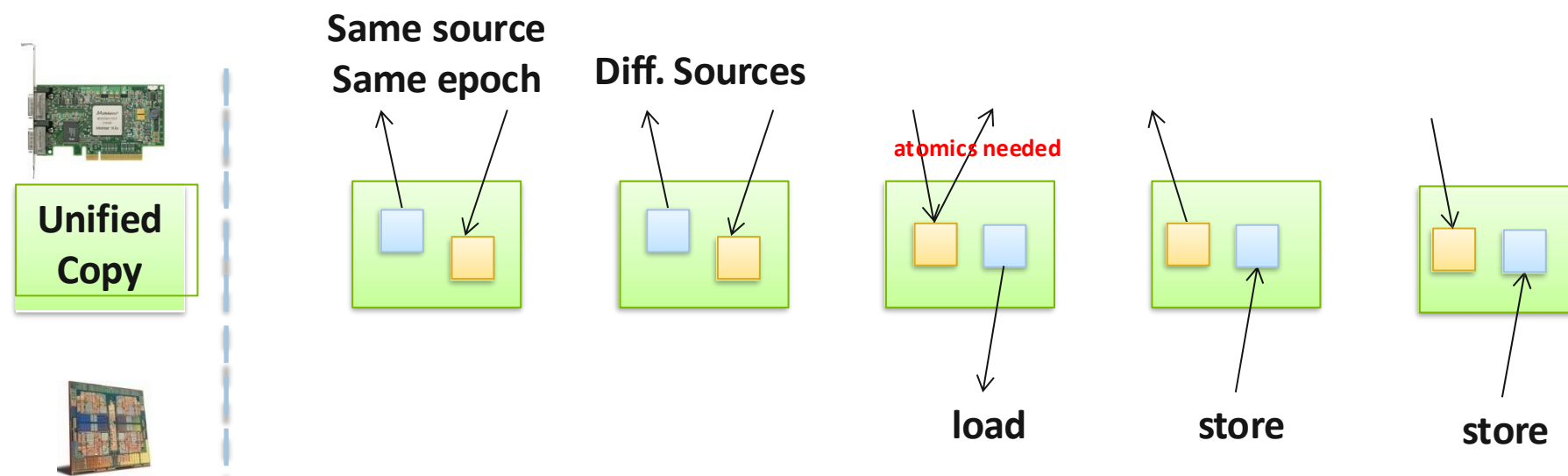


MPI RMA Memory Model (separate windows)



- Very portable, compatible with non-coherent memory systems
- Limits concurrent accesses to enable software coherence

MPI RMA Memory Model (unified windows)



- Allows concurrent local/remote accesses
- Concurrent, conflicting operations are allowed (not invalid)
 - Outcome is not defined by MPI (defined by the hardware)
- Can enable better performance by reducing synchronization

MPI RMA Operation Compatibility (Separate)

	Load	Store	Get	Put	Acc
Load	OVL+NOVL	OVL+NOVL	OVL+NOVL	NOVL	NOVL
Store	OVL+NOVL	OVL+NOVL	NOVL	X	X
Get	OVL+NOVL	NOVL	OVL+NOVL	NOVL	NOVL
Put	NOVL	X	NOVL	NOVL	NOVL
Acc	NOVL	X	NOVL	NOVL	OVL+NOVL

This matrix shows the compatibility of MPI-RMA operations when two or more processes access a window at the same target concurrently.

OVL – Overlapping operations permitted

NOVL – Nonoverlapping operations permitted

X – Combining these operations is OK, but data might be garbage

MPI RMA Operation Compatibility (Unified)

	Load	Store	Get	Put	Acc
Load	OVL+NOVL	OVL+NOVL	OVL+NOVL	NOVL	NOVL
Store	OVL+NOVL	OVL+NOVL	NOVL	NOVL	NOVL
Get	OVL+NOVL	NOVL	OVL+NOVL	NOVL	NOVL
Put	NOVL	NOVL	NOVL	NOVL	NOVL
Acc	NOVL	NOVL	NOVL	NOVL	OVL+NOVL

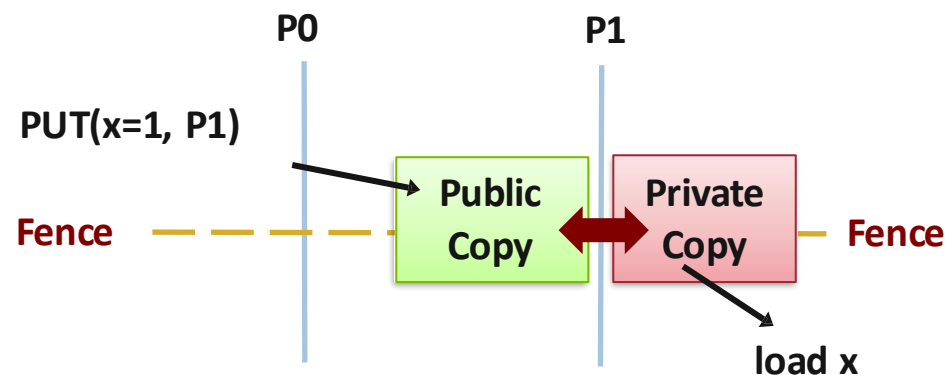
This matrix shows the compatibility of MPI-RMA operations when two or more processes access a window at the same target concurrently.

OVL – Overlapping operations permitted

NOVL – Nonoverlapping operations permitted

Synchronizing Local and RMA Access (1/2)

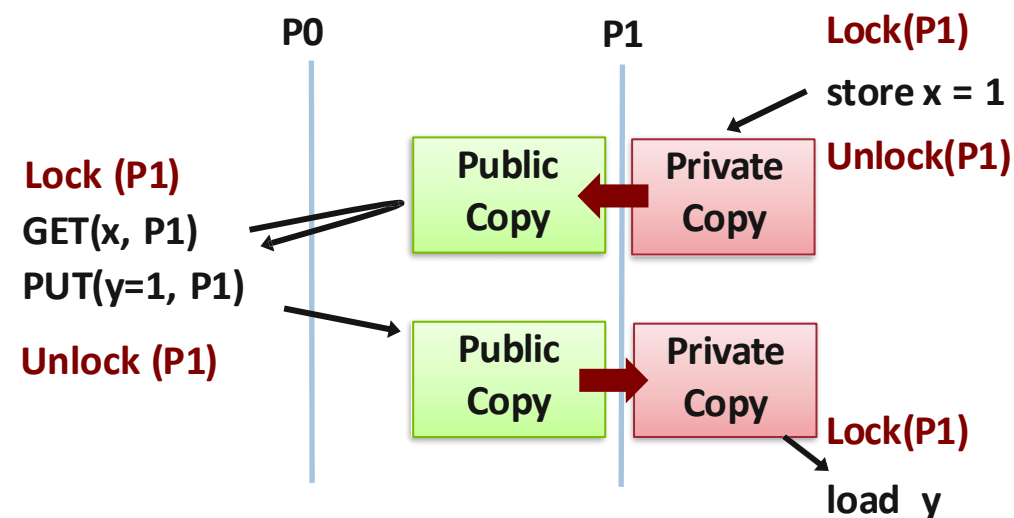
- RMA operations access the public copy of window
- Local load/store update the private copy
 - Including using as MPI send/receive buffers
- Ensure memory synchronization for portability
- Implicit memory synchronization (i.e., memory barrier) in RMA synchronization calls
 - Fence: Synchronize private and public copies of local window



Synchronizing Local and RMA Access (2/2)

■ Passive target epoch

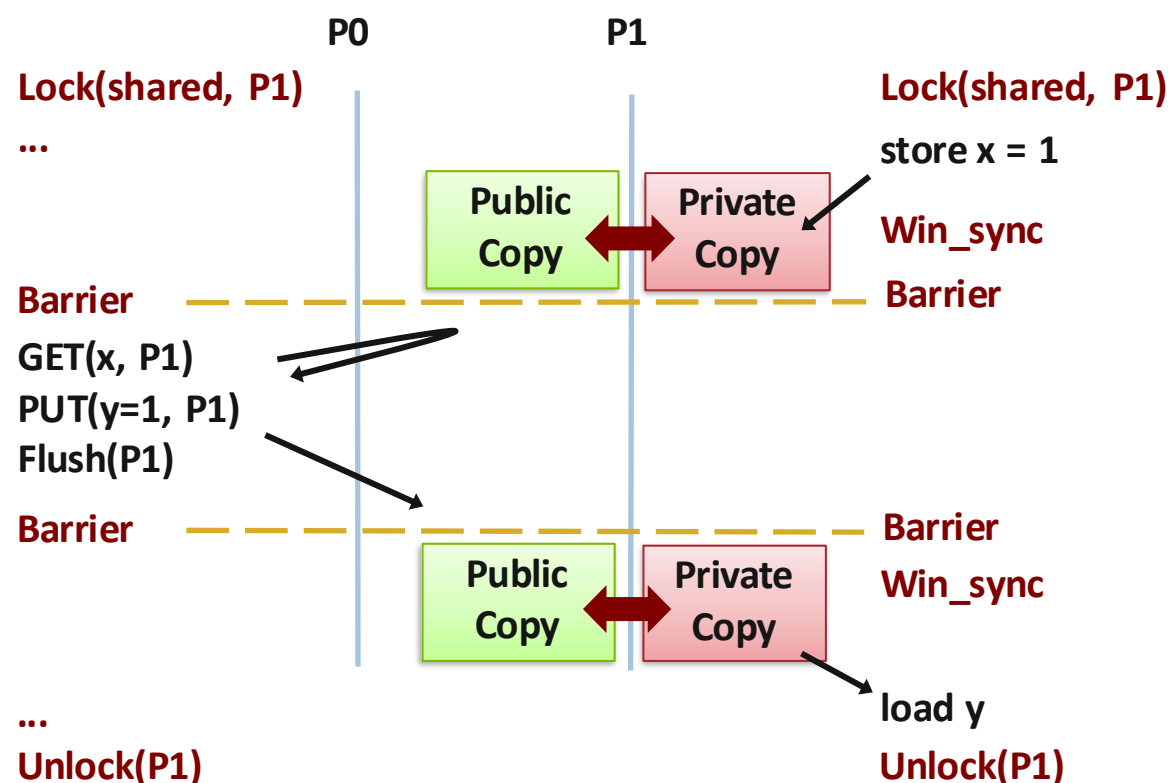
- Lock/Lock_all: Updates in public copy becomes visible in private
- Unlock/Unlock_all: Updates in private copy becomes visible in public



Window synchronization: MPI_WIN_SYNC

```
MPI_Win_sync(MPI_Win win)
```

- Synchronizes the public and private copies of local window in passive target epoch

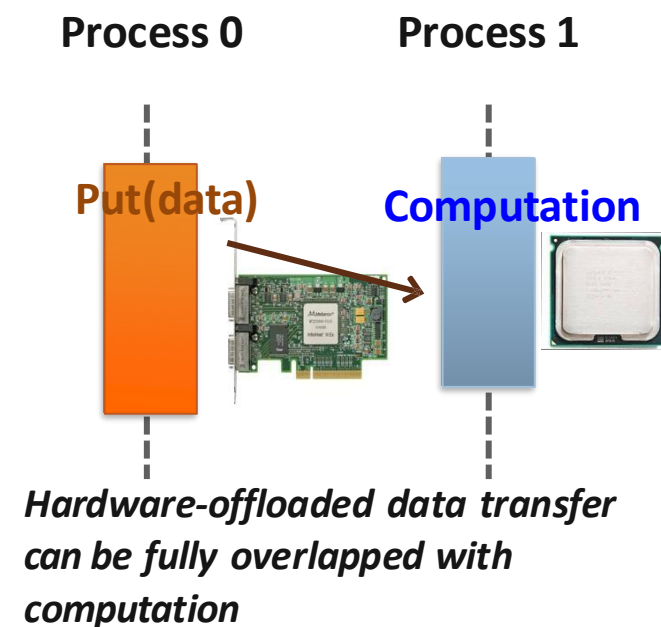


Exercise 3: Stencil with RMA Lock_all/Unlock_all (PUT model)

- In the fence version of the stencil code, RMA synchronization involves the target processes
- Let's try to use RMA Lock_all/Flush_all/Unlock_all
 - Only the origin processes call RMA synchronization
 - Still need Barrier for process synchronization (e.g., ensure neighbors have completed data update to my local window)
 - Need Win_sync for memory synchronization
- Start from rma/stencil_fence_put.c
- Solution available in rma/stencil_lock_put.c

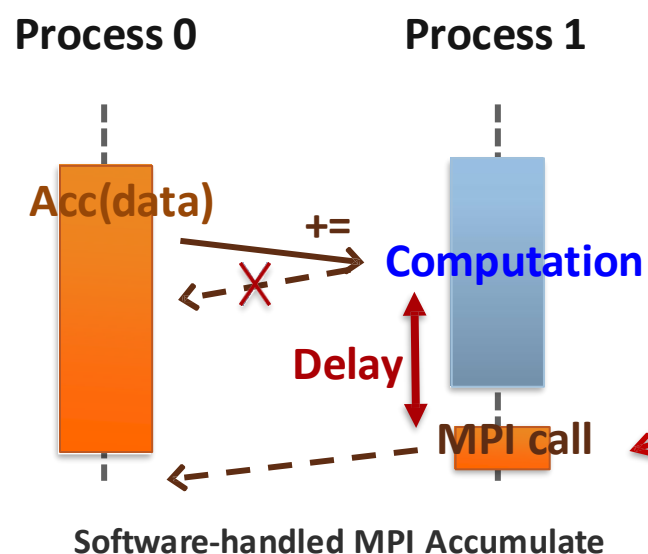
Hardware-Offloaded Communication

- Offloading data transfer to hardware is ideal for performance
 - Two-sided (e.g., SEND/RECEIVE):
 - Require complex message matching (rank + tag + comm), especially for wildcard receives (MPI_ANY_TAG|ANY_SOURCE)
 - Supported HW: Mellanox ConnectX-5 (support HW tag matching)
 - One-sided (e.g., PUT/GET/ACCUMUALTE):
 - No matching requirement, easier for hardware offloading
 - Natively supported on various RDMA networks such as Mellanox InfiniBand, Cray Aries, and Fujitsu Tofu



Asynchronous Execution of MPI RMA

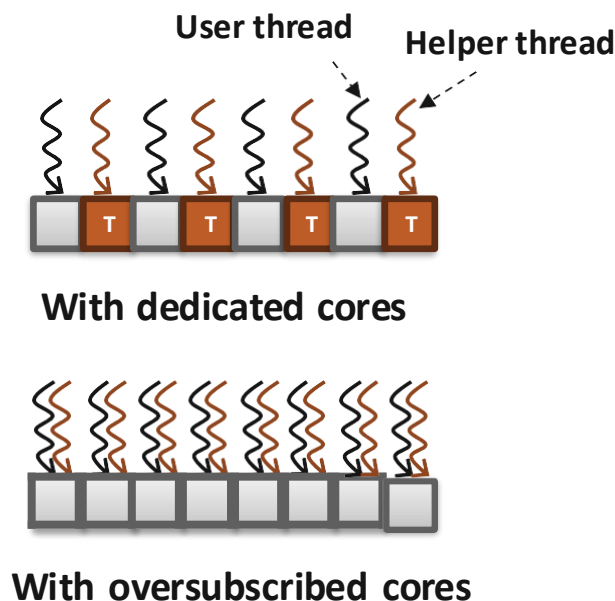
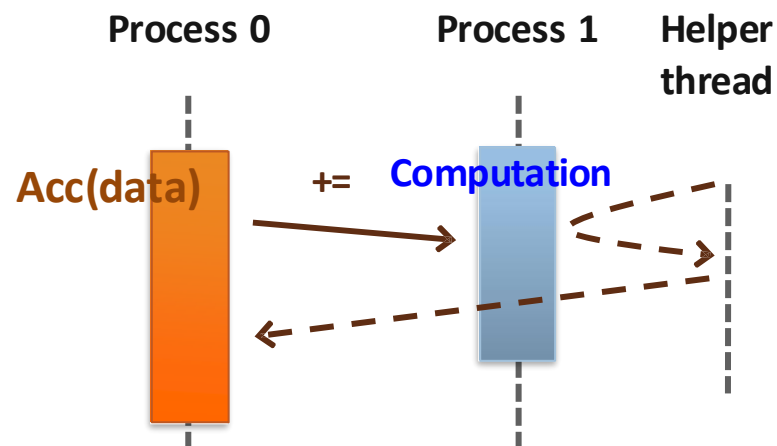
- Asynchronous execution of RMA depends on the MPI implementation, which in turn depends on what the network hardware provides
- Most common situation on current network hardware:
 - Some operations are natively supported in hardware (e.g., contiguous PUT/GET)
 - Other operations need to be emulated in software



Software implementation of one-sided operations means that **the target process has to run MPI code to make progress**. This may require the target process to make an MPI call (more later)

Possible Solution 1: Thread-based Progress (1/2)

- Every MPI process creates a dedicated helper thread at `MPI_Init_thread` or `MPI_Init`
 - Supported by most MPI implementations, but unlikely to be default
 - Might need to turn on some environment variable (check the documentation)
- The thread polls MPI progress for the process while the process is computing
- Dedicates some number of computing cores per process
- Multithreading safety overhead (e.g., MPI internal lock contention between threads, memory barriers)



Possible Solution 1: Thread-based Progress (2/2)

- Available in many mainstream MPI implementations
- Core binding is important! See vendor documentation

MPICH:

```
% export MPICH_CVAR_ASYNC_PROGRESS=1
```

Intel MPI:

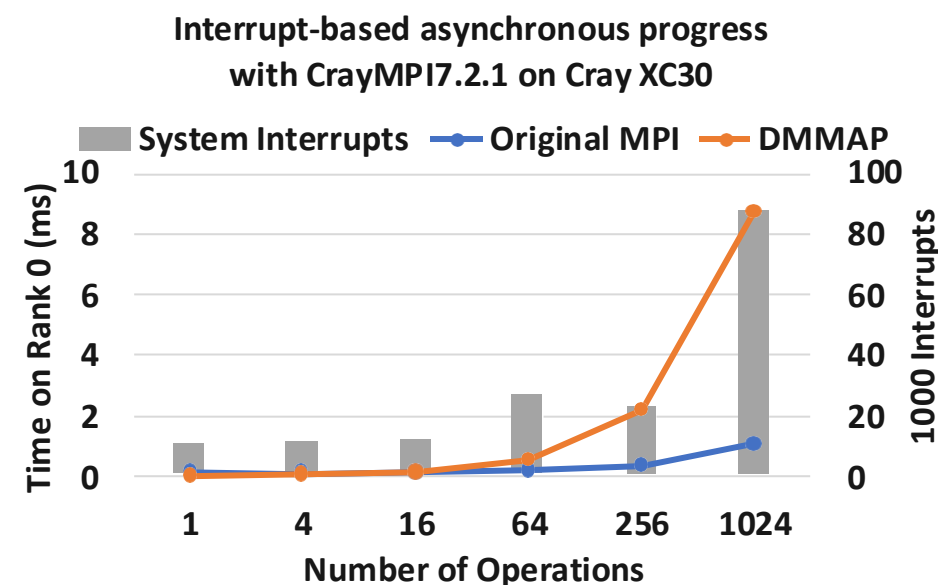
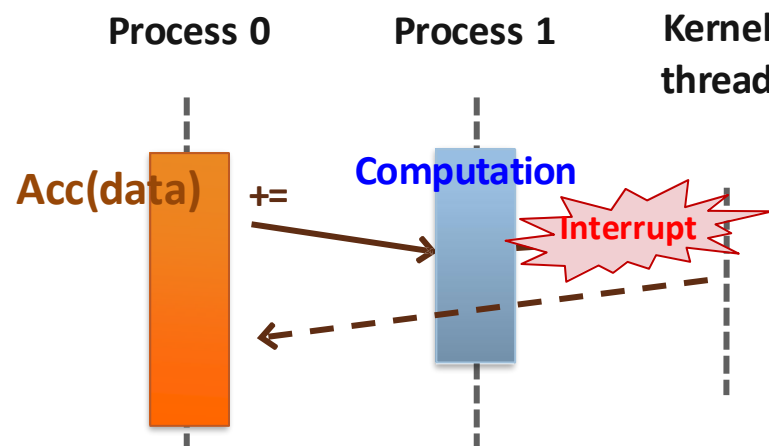
```
% export I_MPI_ASYNC_PROGRESS=1
```

Cray MPI:

```
% export MPICH_NEMESIS_ASYNC_PROGRESS=1  
% export MPICH_MAX_THREAD_SAFETY=multiple
```

Possible Solution 2: Interrupt-based Progress

- Utilize hardware interrupts to awaken a kernel thread when new message arrives
- Examples: Cray MPI DMAPP mode (**deprecated from v7.6.0**), IBM MPI on Blue Gene/P
- Overhead of frequent interrupts, need special hardware support
 - Not a common model for most current networks



Section Summary

- MPI one-sided communication is associated with windows
- Operations include basic PUT, GET, and Atomic operations
- Synchronization modes
 - Active-target (similar to two-sided) : FENCE
 - Passive-target: LOCK-UNLOCK, FLUSH, FLUSH_LOCAL...
- Enable asynchronous progress for performance

